

La Cave du Coin — Mise en place de l'environnement de test (staging)

Journal détaillé, commande par commande, de la création d'un environnement ISO prod et de son accès via pgAdmin — pour comprendre et pouvoir refaire seul à l'avenir.

1. Objectif : pourquoi cet environnement ?

Le besoin de départ : disposer d'un endroit où exécuter des requêtes SQL sur des données réalistes, sans jamais risquer d'abîmer la base de production. La solution retenue est un **second stack applicatif complet** (application + base PostgreSQL), tournant sur le **même serveur (VPS)** que la production mais dans des conteneurs Docker totalement séparés, avec sa propre base de données. Une fois par mois, un script recopie le code et les données de production dans cet environnement, pour qu'il reste "ISO prod" (identique) dans le temps.

2. Vue d'ensemble de l'architecture obtenue

Un seul serveur (VPS Hetzner) héberge deux "stacks" Docker Compose indépendants, identifiés par un nom de projet différent (-p staging pour le second) :

- **Stack "prod"** (celui déjà en place) : conteneurs app et db, plus caddy (reverse proxy HTTPS) qui reçoit tout le trafic public sur les ports 80/443 et le redirige vers app. C'est ce qui sert `lacaveducoin.com`.
- **Stack "staging"** (nouveau) : ses propres conteneurs db et app (une base et une application indépendantes), plus pgadmin (interface web pour interroger la base). Rien n'est exposé publiquement par défaut : les ports de la base/app sont liés uniquement à `127.0.0.1` (la boucle locale du serveur), donc injoignables depuis internet même en cas d'oubli de pare-feu.

Pour se connecter à la base staging, deux chemins possibles : un **tunnel SSH** depuis un PC non bridé (redirige un port du PC vers le port local du VPS), ou **pgAdmin en HTTPS** via un sous-domaine dédié (`pgadmin.lacaveducoin.com`) — utile quand le poste utilisé bloque les connexions SSH "brutes" (cas du PC professionnel Allianz, protégé par un proxy Zscaler qui laisse passer le HTTPS vers des domaines reconnus mais bloque le SSH direct).

3. Nouveaux fichiers ajoutés au projet

Chaque commande exécutée sur le serveur s'appuie sur des fichiers ajoutés dans le dépôt Git, récupérés simplement via `git pull` — c'est un principe important : on évite de taper du contenu compliqué directement dans la console (voir section 6), on le prépare dans des fichiers versionnés à l'avance.

Commande / action	Ce que ça fait et pourquoi
<code>docker-compose.staging.yml</code>	Fichier Docker Compose "overlay" (complément) qui ajoute au fichier de base : les ports 127.0.0.1 pour db/app, le service pgadmin, et le rattachement au réseau partagé caddy-net.
<code>.env.staging.example</code>	Modèle de configuration pour le stack staging (mots de passe, secrets). À copier en <code>.env.staging</code> puis à personnaliser — jamais commité tel quel dans Git (fichier réel exclu via <code>.gitignore</code>).
<code>scripts/refresh_staging_from_prod.sh</code>	Script principal : synchronise le code (<code>git pull</code>), reconstruit/démarre le stack staging, puis copie les données de prod vers staging. Utilisé pour le premier lancement ET pour le rafraîchissement mensuel automatique.
<code>scripts/verify_staging.sh</code>	Petit script de vérification : affiche les utilisateurs et produits présents dans la base staging, pour confirmer que la copie de données a fonctionné.

<code>deploy/staging-refresh.cron</code>	Contenu prêt à l'emploi pour <code>/etc/cron.d/</code> : programme l'exécution automatique du script de rafraîchissement le 1er de chaque mois à 2h du matin.
<code>scripts/add_pgadmin_env_vars.sh</code> / <code>add_pgadmin_domain.sh</code>	Scripts qui ajoutent automatiquement, sans risque de faute de frappe, les lignes de configuration nécessaires à pgAdmin dans <code>.env.staging</code> et <code>.env (prod)</code> .
Caddyfile (modifié)	Ajout d'un second "site" géré par Caddy : le sous-domaine pgAdmin, routé vers le conteneur pgadmin plutôt que vers l'application principale.
<code>docker-compose.vps.yml</code> (modifié)	Ajout du rattachement de Caddy au réseau partagé caddy-net, pour qu'il puisse atteindre le conteneur pgadmin qui vit dans l'autre stack (staging).

4. Déroulé complet, commande par commande

4.1 — Récupérer le code et préparer la configuration

Commande / action	Ce que ça fait et pourquoi
<code>git pull</code>	Télécharge et applique les derniers fichiers du dépôt GitHub sur le serveur — c'est ainsi que tous les nouveaux fichiers (scripts, Caddyfile, etc.) arrivent sur le VPS, sans jamais les taper à la main.
<code>docker network create caddy-net</code>	Crée un réseau Docker "nommé" et partagé. Par défaut, chaque projet Docker Compose crée son propre réseau isolé (les conteneurs "prod" ne voient pas les conteneurs "staging"). En créant ce réseau à part et en y rattachant à la fois Caddy (côté prod) et pgAdmin (côté staging), les deux peuvent se parler malgré leurs stacks séparés.
<code>cp .env.staging.example .env.staging</code>	Copie le modèle de configuration en un vrai fichier de configuration pour le stack staging. "cp" = copy : duplique un fichier.
<code>nano .env.staging</code>	Ouvre un éditeur de texte en ligne de commande pour renseigner les vraies valeurs (mot de passe de la base, secret de sécurité) à la place des valeurs d'exemple.

4.2 — Premier démarrage du stack staging

Commande / action	Ce que ça fait et pourquoi
<code>chmod 755 scripts/refresh_staging_from_prod.sh</code>	"chmod" change les permissions d'un fichier. 755 signifie : le propriétaire peut lire/écrire/exécuter, les autres peuvent seulement lire/exécuter. Sans ce réglage, Linux refuse de lancer un fichier <code>.sh</code> comme un programme (message "Permission denied").

<code>./scripts/refresh_staging_from_prod.sh</code>	Lance le script. En interne, il exécute dans l'ordre : (1) git pull, (2) "docker compose ... up -d --build" pour démarrer/mettre à jour les conteneurs du stack staging (crée le réseau, le volume, construit l'image de l'app, démarre db/app/pgadmin), (3) une commande combinant pg_dump (côté prod) et pg_restore (côté staging) pour copier l'intégralité de la base de production dans la base staging.
<code>docker exec staging-db-1 psql -U cave_du_coin -c "select ..."</code>	"docker exec" lance une commande À L'INTÉRIEUR d'un conteneur déjà démarré (ici staging-db-1). "psql" est le client en ligne de commande de PostgreSQL ; "-U" précise l'utilisateur, "-c" fait exécuter une requête SQL ponctuelle sans ouvrir de session interactive. Utilisé ici pour vérifier que les données sont bien présentes.

4.3 — Planifier le rafraîchissement automatique mensuel

Commande / action	Ce que ça fait et pourquoi
<code>cp deploy/staging-refresh.cron /etc/cron.d/staging-refresh</code>	Place le fichier de planification dans le dossier que le service cron (le planificateur de tâches de Linux) surveille automatiquement. Chaque fichier dans /etc/cron.d/ contient une ou plusieurs lignes au format : minute heure jour-du-mois mois jour-semaine utilisateur commande. Ici : "0 2 1 * *" = à 0 minute, 2h, le 1er jour de chaque mois, tous mois, tout jour de la semaine.
<code>systemctl status cron</code>	Affiche l'état du service cron lui-même (démarré ou non). "systemctl" est l'outil standard pour gérer les services système sous Linux moderne (Ubuntu, Debian...). Vérifié "active (running)", donc les tâches planifiées seront bien exécutées.
<code>cat /etc/cron.d/staging-refresh</code>	"cat" affiche le contenu d'un fichier. Utilisé pour vérifier que la copie n'a pas été déformée (la console a un bug clavier qui peut altérer certains caractères tapés à la main, cf. section 6) — une simple copie de fichier via "cp", elle, ne risque rien.

4.4 — Accès pgAdmin sans tunnel SSH (HTTPS)

Commande / action	Ce que ça fait et pourquoi
Enregistrement DNS de type A	Ajouté dans l'interface de gestion DNS du domaine (pas sur le serveur) : associe le nom "pgadmin.lacaveducoin.com" à l'adresse IP du VPS. Sans ça, ce nom de domaine ne "pointerait" vers rien.
<code>./scripts/add_pgadmin_env_vars.sh</code>	Ajoute automatiquement au fichier .env.staging les identifiants de connexion à l'interface pgAdmin (email + mot de passe), sans risquer une faute de frappe sur le caractère "@".
<code>./scripts/add_pgadmin_domain.sh</code>	Ajoute automatiquement au fichier .env (celui de production, lu par Caddy) la ligne PGADMIN_DOMAIN=pgadmin.lacaveducoin.com, sans risquer une faute de frappe sur le caractère "_".

<code>docker compose up -d --build</code> (à nouveau, pour la prod)	Relit le <code>.env</code> de production mis à jour et redémarre les conteneurs concernés — ici, essentiellement Caddy, qui prend en compte la nouvelle route vers pgAdmin. "-d" = detached (en arrière-plan), "--build" = reconstruit l'image si le code de l'app a changé (sans effet ici puisque seul Caddy est concerné).
<code>docker compose logs caddy --tail 30</code>	Affiche les 30 dernières lignes de journal (logs) du conteneur Caddy — utilisé pour confirmer que Caddy a bien obtenu un certificat HTTPS valide (Let's Encrypt) pour le nouveau sous-domaine, via le protocole ACME (validation automatique du nom de domaine).

4.5 — Connexion finale dans l'interface pgAdmin

Cette dernière étape se fait entièrement dans le navigateur (pas dans la console du serveur) : ouverture de `https://pgadmin.lacaveducoin.com`, connexion avec l'email/mot de passe pgAdmin, puis "Register → Server" pour enregistrer la base staging elle-même comme cible de requêtes (host = db — le nom du service dans `docker-compose.yml`, résolu automatiquement par Docker — port 5432, base `cave_du_coin`, utilisateur `cave_du_coin`, mot de passe = celui de `POSTGRES_PASSWORD` dans `.env.staging`).

5. Le bug clavier de la console Hetzner

La console web du VPS (utilisée en l'absence d'un accès SSH direct fonctionnel) déforme certains caractères nécessitant la touche Shift à la frappe : `+` devient `=`, `_` devient `-`, `( * )` se transforment en chiffres, etc. Ce n'est pas systématique ni prévisible à 100%, donc la parade a été, chaque fois que possible :

- Préparer le contenu compliqué (scripts, fichiers de configuration, lignes cron) **dans des fichiers versionnés sur GitHub**, récupérés via `git pull` ou copiés via `cp` — aucun caractère n'est alors tapé à la main.
- Utiliser la **complétion automatique (touche Tab)** pour les noms de fichiers contenant des underscores : le terminal insère lui-même les caractères exacts, sans passer par le clavier physique pour cette partie.
- Préférer des variantes de commandes sans caractère à risque quand elles existent (`chmod 755` au lieu de `chmod +x`).
- Toujours **vérifier avec cat** le contenu d'un fichier après une opération sensible, avant de l'utiliser.

6. Concepts clés à retenir

Commande / action	Ce que ça fait et pourquoi
<code>Docker Compose "projet"</code>	Un ensemble de conteneurs démarrés ensemble depuis un ou plusieurs fichiers <code>docker-compose*.yml</code> partagent un nom de "projet" (par défaut, le nom du dossier ; ici forcé à "staging" via <code>-p staging</code>). Chaque projet a son propre réseau et ses propres volumes par défaut, isolés des autres projets — d'où la nécessité d'un réseau partagé explicite (<code>caddy-net</code>) pour relier deux projets.
Réseau Docker & résolution de noms	À l'intérieur d'un même réseau Docker, chaque service est joignable par son nom (ex: "db", "pgadmin") comme s'il s'agissait d'une adresse — Docker gère cette résolution automatiquement (DNS interne), pas besoin de connaître d'adresse IP.
Ports liés à <code>127.0.0.1</code>	Une règle <code>"127.0.0.1:PORT:PORT"</code> dans <code>docker-compose</code> expose un service uniquement sur l'interface "boucle locale" du serveur (accessible seulement depuis le serveur lui-même), jamais depuis internet — une protection au niveau réseau, indépendante du pare-feu.

<code>pg_dump / pg_restore</code>	Deux outils PostgreSQL complémentaires : <code>pg_dump</code> exporte le contenu d'une base (structure + données) dans un fichier ; <code>pg_restore</code> le réimporte. L'option <code>--clean --if-exists</code> fait d'abord table rase de la base de destination avant de tout réimporter, pour garantir une copie fidèle et non un simple ajout aux données existantes.
Reverse proxy (Caddy) & Let's Encrypt	Caddy reçoit tout le trafic HTTPS public et le redirige ("reverse proxy") vers le bon conteneur selon le nom de domaine demandé. Il obtient et renouvelle automatiquement de vrais certificats HTTPS gratuits auprès de Let's Encrypt, via le protocole ACME, sans aucune configuration manuelle de certificat.
Tunnel SSH (port forwarding)	Une commande <code>"ssh -L port_local:127.0.0.1:port_distant"</code> ouvre un tunnel chiffré : tout ce qui est envoyé sur le port local de votre PC est transporté jusqu'au serveur puis redirigé vers un port qui, sur le serveur, n'est accessible que localement. Permet d'atteindre un service "privé" du serveur sans jamais l'exposer publiquement.

7. Ce qui est désormais automatique vs. ce qui reste manuel

Automatique : le rafraîchissement mensuel (code + données) via le cron planifié — aucune action nécessaire, un journal est conservé dans `staging_refresh.log`. Le renouvellement des certificats HTTPS (Caddy s'en occupe seul, bien avant leur expiration).

Manuel (rare, seulement si le besoin évolue) : ajouter un nouvel enregistrement DNS si un nouveau sous-domaine est nécessaire un jour ; relancer `docker compose up -d --build` après un `git pull` si le code du projet évolue à nouveau (comme pour la production) ; changer les mots de passe par défaut si ce n'est pas déjà fait.

■■ Rappel sécurité : chaque rafraîchissement copie les vraies données clients de production (noms, téléphones, adresses, soldes) dans l'environnement de test. Comme il n'y a pas de tiers impliqué (votre propre activité), ce n'est pas un problème de conformité en soi — gardez simplement le même niveau de vigilance sur les mots de passe de cet environnement que sur la production.